

MFA (Multi-Factor Authentication) Testing Strategy – Automation Framework

 Page is about?

 Purpose:

*This documentation for **automated testing strategies for Multi-Factor Authentication (MFA)** (such as OTP via SMS for UE/IDP Login flows), covering both **UI-based automation** and **API-based OTP extraction** methods, using your current Selenium-based framework.*

Problem Statement

Universal Enrollment and IDP flows require entering a One-Time Password (OTP) (*this is mandatory step*) sent to the user's mobile device. Manual test cases currently **bypass or disable MFA** for select test accounts.

Goal

Develop a sustainable automation strategy to validate MFA flows **organically**, while maintaining CI/CD compatibility.

Section 1: Challenges in Automating OTP Flows

Challenge	Impact
OTP sent to real device	Not accessible via automation; cannot be retrieved in real-time
OTP expiry window	Short-lived (e.g., 60 seconds) limits timing accuracy
CAPTCHA or reCAPTCHA	May block automated flows unless bypassed
External Dependency (SMS)	Automation suite lacks SMS gateway access
Non-idempotent Behavior	OTP changes per request; cannot use static data

Section 2: Recommended Automation Strategies

Strategy A: Test Account Whitelisting (Current Temporary Method)

- Disable MFA for QA test accounts.
- Test flows as if MFA is passed.
- *Simple, low setup.*
- *Not a real validation of OTP logic.*

Strategy B: OTP Interception via API or DB (Ideal if Available)

Check with Dev Team if OTPs are generated and logged in backend (DB or exposed API)

Implementation Options:

1. OTP Retrieval via API Endpoint

- Intercept OTP via REST API call before UI input.
- Store OTP to a variable and input in Selenium form.
- Sample Flow:

```
String otp = RestAssured
    .given().queryParams("userId", "testuser123")
    .when().get("/otp-service/latest")
    .then().extract().path("otp");
driver.findElement(By.id("otpField")).sendKeys(otp);
```

2. OTP Retrieval via Database Query

- Fetch latest OTP from backend DB:
e.g. `SELECT otp_code FROM mfa_audit WHERE user_id = 'testuser123' ORDER BY created_at DESC LIMIT 1;`
- Use JDBC connection in test and inject value into form.

3. Webhook or Log File Monitoring

- Monitor logs or webhook events if SMS or OTP is triggered asynchronously.
- Use test server with webhook simulator or logging endpoint.

Strategy C: Use Mock OTP Middleware in Lower Environments

- Replace OTP sender module in **STAGE/QA** env with mock service:
 - Always send OTP = 123456
 - Or return via test-only interface
- Useful for **controlled regression validation** with CI/CD

Section 3: How to Implement via Existing Automation Framework

Option 1: Integrate API Call in Test Flow (if OTP API exists)

- Use **RestAssured** to call OTP API.
- Store and assert OTP field using Selenium.
- Add retry in case of delay.

Option 2: JDBC OTP Fetch Logic (via Oracle DB)

- Extend your data provider or helper class to:
 - Connect to DB using JDBC
 - Query OTP table
 - Pass OTP as test data
- Example:

```
String otp = DBUtil.getLatestOTP("testuser123");
sendKeys(driver.findElement(By.id("otpField")), otp);
```

Option 3: Hybrid - Switch Based on Env

```
if (env.equals("QA")) {
    // get OTP from API or DB
} else {
    // use static OTP for mock service
}
```

Option 4: Bypass MFA Flow for Non-Regression Testing

- Tag scenarios like @NoMFA
- Skip OTP steps if not part of current suite
- Useful during smoke or CRUD validation

Section 4: Additional Recommendations

Area	Best Practice
OTP Logging	Ask Dev to log OTPs for all test users in a centralized, queryable table/log/API
Expiry Timing	Add retry logic with 2–3 attempts to handle delay in OTP delivery
Concurrency	Ensure test OTP generation is tied to a dedicated account/test context
Alerting	Track OTP mismatch or failure via test report screenshot and error logs
Security	Do NOT test OTPs on production unless authorized and non-intrusive

Final Plan Summary

Strategy	Ideal Use Case	Effort	CI/CD Friendly
API-based OTP Retrieval	When OTP service exposes endpoint		Yes
DB Query for OTP	If OTP stored in DB and accessible		Yes
Static OTP Mocking	Lower environments only		Yes
Test User Whitelisting	Quick local validation		No

